SYSTEMS & DEVICES 3 (SYS3 - COM00018I) FORMATIVE EXERCISE 2 SOLUTIONS

Covering Control Hazards, Branch Prediction, Static and Dynamic Scheduling, Superscalar Processors.

Please try to complete it by 27/03/2026 (Fri, S2 Week 7). We will use the Lecture Review session on 16/04/2026 (Thu, S2 Week 8) for discussions if there are any questions on it.

Q1. Explain what is Delayed Branching?

Solution:

One way to maximize the use of the pipeline, is to find an instruction that can be safely executed whether the branch is taken or not, and execute that instruction. So, when a branch instruction is encountered, the hardware puts the instruction following the branch into the pipe and begins executing it, just as in predict-not-taken. However, unlike in predict-not taken, we do not need to worry about whether the branch is taken or not, we do not need to clear the pipe because no matter whether the branch is taken or not, we know the instruction is safe to execute.

Q2. True or False: In a pipelined processor, control hazard penalties can be eliminated by dynamic branch prediction.

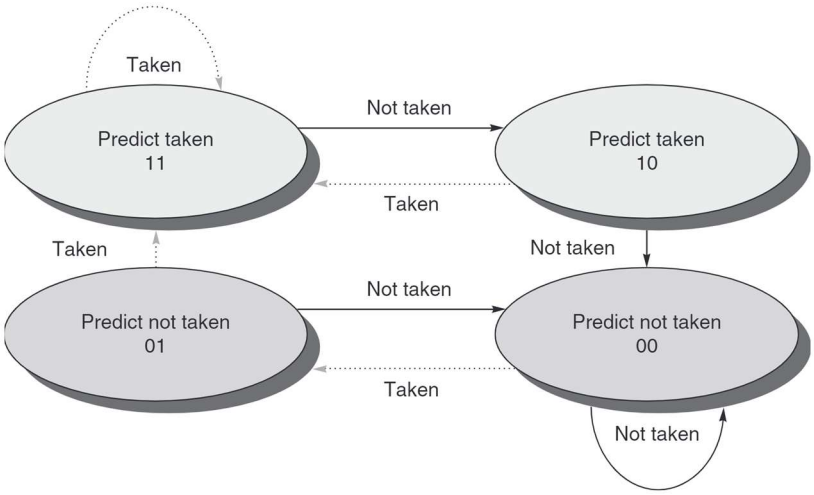
Solution:

False, it cannot completely eliminate, though it can reduce Control Hazard Penalties.

Q3. We have a program core consisting of five conditional branches. The program core will be executed thousands of times. Below are the outcomes of each branch for one execution of the program core (T for taken, N for not taken).
Branch 1: T-T-T-T
Branch 2: N-N-N-N
Branch 3: N-T-N-T-N-T
Branch 4: T-N-T-N-T-N
Branch 5: T-T-N-T-T-N-T-T

Assume the behaviour of each branch remains the same for each program core execution. For the dynamic branch prediction schemes below, assume that each branch maps to a unique entry in the branch history table. Further assume that each BHT entry is initialized to the same stage before each execution

(a) Always taken
(b) Always not taken
(c) 1-bit predictor, initialized to predict taken
(d) 2-bit predictor (Finite State Machine as shown below), initialized to predict taken (10)



What is the branch prediction accuracy for branch 1 to 5 with prediction scheme (a) through (d)? Use the following table to fill out your final answer.

Solutions:

Prediction results:

Branches	Prediction Schemes			
	(a)	(b)	(c)	(d)
1	T-T-T-T T-T-T-T	T-T-T-T N-N-N-N	T-T-T-T T-T-T-T	T-T-T-T T-T-T-T
2	N-N-N-N T-T-T-T	N-N-N-N N-N-N-N	N-N-N-N T-N-N-N	N-N-N-N T-N-N-N
3	N-T-N-T-N-T T-T-T-T-T-T	N-T-N-T-N-T N-N-N-N-N-N	N-T-N-T-N-T T-N-T-N-T-N	N-T-N-T-N-T T-N-N-N-N-N
4	T-N-T-N-T-N T-T-T-T-T-T	T-N-T-N-T-N N-N-N-N-N-N	T-N-T-N-T-N T-T-N-T-N-T	T-N-T-N-T-N T-T-T-T-T-T
5	T-T-N-T-T-N-T-T T-T-T-T-T-T-T-T	T-T-N-T-T-N-T-T N-N-N-N-N-N-N-N	T-T-N-T-T-N-T-T T-T-T-N-T-T-N-T	T-T-N-T-T-N-T-T T-T-T-T-T-T-T-T

Accuracies:

Branches	Prediction Schemes			
	(a)	(b)	(c)	(d)
1	100%	0%	100%	100%
2	0%	100%	75%	75%
3	50%	50%	0%	33.3%
4	50%	50%	16.7%	50%
5	75%	25%	50%	75%

Q4. What is the shortcoming of Tomasulo's algorithm that leads to the introduction of speculation? How is the reorder buffer used to introduce speculation?

Solutions:

Without speculation, any conditional branch will prevent a processor from issuing instructions predicted to follow the branch, since those instructions cannot be allowed to complete until we are sure that they will execute.

The reorder buffer is a queue of speculated results. Upon completion of an instruction's computation, the result is written into the reorder buffer rather than directly to the instruction's destination register. The value is moved from the reorder buffer to the destination register only once branch predictions preceding the instruction are confirmed.

Q5. (True or False) In superscalar mode, all the similar instructions are grouped and executed together.

Solutions:

False.

In superscalar mode, multiple instructions from the instruction stream can be issued and executed simultaneously, but not necessarily all similar instructions are grouped and executed together. Instead, the processor dynamically identifies independent instructions from different parts of the program and schedules them for execution in parallel if there are enough execution units available. This includes instructions from different categories such as arithmetic, load/store, branch, and others.

The objective of superscalar execution is to maximize instruction throughput by exploiting instruction-level parallelism within the program. This means that the processor can execute multiple instructions concurrently, even if they are not of the same type or category. Therefore, while some similar instructions may be executed together if they are independent and there are sufficient resources, the main focus is on executing as many instructions in parallel as possible, regardless of their similarity.

Q6. In super-scalar processors, out of order mode of execution is used, the results are stored in _____?

a) ROBs

b) Special memory locations

c) Temporary registers

d) TLB

Solutions:

a).

In superscalar processors, the out-of-order execution mode allows instructions to be executed as soon as their operands are available, regardless of their original program order. This means that instructions can be completed and produce results out of program order. To handle out-of-order execution and ensure correct program semantics, superscalar processors typically employ a structure known as the reorder buffer (ROB). The ROB serves as a temporary storage mechanism for the results produced by instructions as they complete execution.

Q7. A special unit used to govern the out of order execution of the instructions is called as _____

a) Commitment unit

b) Temporal unit

c) Monitor

d) Supervisory unit

Solutions:

a).

The commitment unit is primarily responsible for ensuring that the results of instructions are committed to the architectural state of the processor in the correct program order.

Once instructions have completed execution, their results are stored in a structure such as the reorder buffer (ROB). The commitment unit then ensures that the results are committed to the architectural state of the processor in the correct program order, based on the information stored in the ROB.

Q8. The commitment unit uses a queue called _____

a) Reorder buffer

b) Commitment buffer

c) Storage buffer

d) None of the mentioned

Solutions:

b).

The commitment unit typically uses a queue called the "commit queue" or "commit buffer."

The commit queue is a structure within the commitment unit that holds the results of instructions until they are ready to be committed to the architectural state of the processor. It ensures that instructions are committed in the correct program order, based on the information stored in the reorder buffer (ROB) or similar structure. As instructions complete execution and their results are stored in the ROB, they are queued in the commit queue. The commitment unit then processes instructions from the commit queue in program order, ensuring that the results are written back to the register file or memory in the correct sequence. The commit queue helps manage the flow of completed instructions through the commitment unit, ensuring precise and orderly updates to the architectural state of the processor while allowing out-of-order execution to proceed efficiently.

Q9. Consider the following instruction sequence:

1. LD R1, 0(R2)

2. ADD R3, R1, R4

3. SUB R5, R3, R6

4. MUL R7, R8, R9

5. ADD R10, R1, R11

Assume: Load latency = 2 cycles; ADD/SUB latency = 1 cycle; MUL latency = 3 cycles; In-order issue pipeline.

Answer the following questions:

a) Identify data dependencies

b) Perform static scheduling to minimize stalls

c) Draw the execution timeline

Solutions:

Identify dependencies

Dependencies:

2 depends on 1 (RAW on R1)

3 depends on 2 (RAW on R3)

5 depends on 1 (RAW on R1)

Instruction 4 is independent.

Original execution (without scheduling)

Cycle	Instruction
1	LD
2	stall
3	ADD
4	SUB
5	MUL
6	ADD

Stalls occur because ADD waits for LD result.

Reorder independent instruction

Move instruction 4 earlier.

Optimized schedule:

1 LD R1,0(R2)

2 MUL R7,R8,R9

3 ADD R3,R1,R4

4 SUB R5,R3,R6

5 ADD R10,R1,R11

Execution timeline

Cycle	Instruction
1	LD
2	MUL
3	ADD
4	SUB
5	ADD

Stall removed using instruction reordering.

Q10. Given the loop:

Loop:

LD F0, 0(R1)

ADD F4, F0, F2

SD F4, 0(R1)

ADDI R1, R1, #8

BNE R1, R3, Loop

Assume: Load latency = 2 cycles; ADD latency = 2 cycles; Store latency = 1 cycle.

Answer the following questions:

a) Unroll the loop 2 times.

b) Apply static scheduling to reduce stalls.

Solutions:

In original loop, dependencies in one iteration:

LD → ADD → SD

ADD needs F0 from LD; SD needs F4 from ADD

So the pipeline has two RAW hazards:

LD → ADD; ADD → SD

Pipeline execution without optimization, timeline example:

Cycle	Instruction
1	LD F0
2	STALL
3	ADD F4
4	STALL
5	SD F4

Two stall cycles appear because: ADD must wait for the loaded value; Store must wait for ADD result.

Unroll the loop

```
LD   F0, 0(R1)
ADD  F4, F0, F2
SD   F4, 0(R1)

LD   F6, 8(R1)
ADD  F8, F6, F2
SD   F8, 8(R1)

ADDI R1, R1, #16
BNE  R1, R3, Loop
```

Now we have more independent instructions.

Reorder instructions

```
LD   F0, 0(R1)
LD   F6, 8(R1)
ADD  F4, F0, F2
ADD  F8, F6, F2
SD   F4, 0(R1)
SD   F8, 8(R1)
ADDI R1, R1, #16
BNE  R1, R3, Loop
```

Key idea:

The second load fills the stall of the first load.

Now loads hide arithmetic latency.

Pipeline Timeline After Optimization:

Cycle	Instruction
1	LD F0
2	LD F6
3	ADD F4
4	ADD F8
5	SD F4
6	SD F8

Now: ADD F4 executes after enough time for LD F0; ADD F8 executes after LD F6; No pipeline bubbles are needed.

We have the benefits of fewer stalls and more instruction-level parallelism (ILP).

Q11. Explain why the following pair cannot issue together:

1 ADD R1,R2,R3

2 SUB R4,R1,R5

Solutions:

Instruction 2 uses R1 produced by instruction 1. This is a RAW hazard.

Superscalar processors must perform dependency checking before issuing multiple instructions in the same cycle. Therefore:

Cycle 1: ADD

Cycle 2: SUB

They cannot issue simultaneously.

Q12. Explain how superscalar processors exploit instruction-level parallelism (ILP) and discuss two major challenges.

Solutions:

Superscalar processors issue multiple instructions per clock cycle using multiple functional units. Example:

2–8 instructions per cycle

They exploit Instruction-Level Parallelism (ILP).

Challenges:

Dependency checking: RAW, WAR, WAW hazards;

Branch prediction: Branches limit parallel instruction fetch;

Instruction window complexity: Hardware must search many instructions for parallelism.